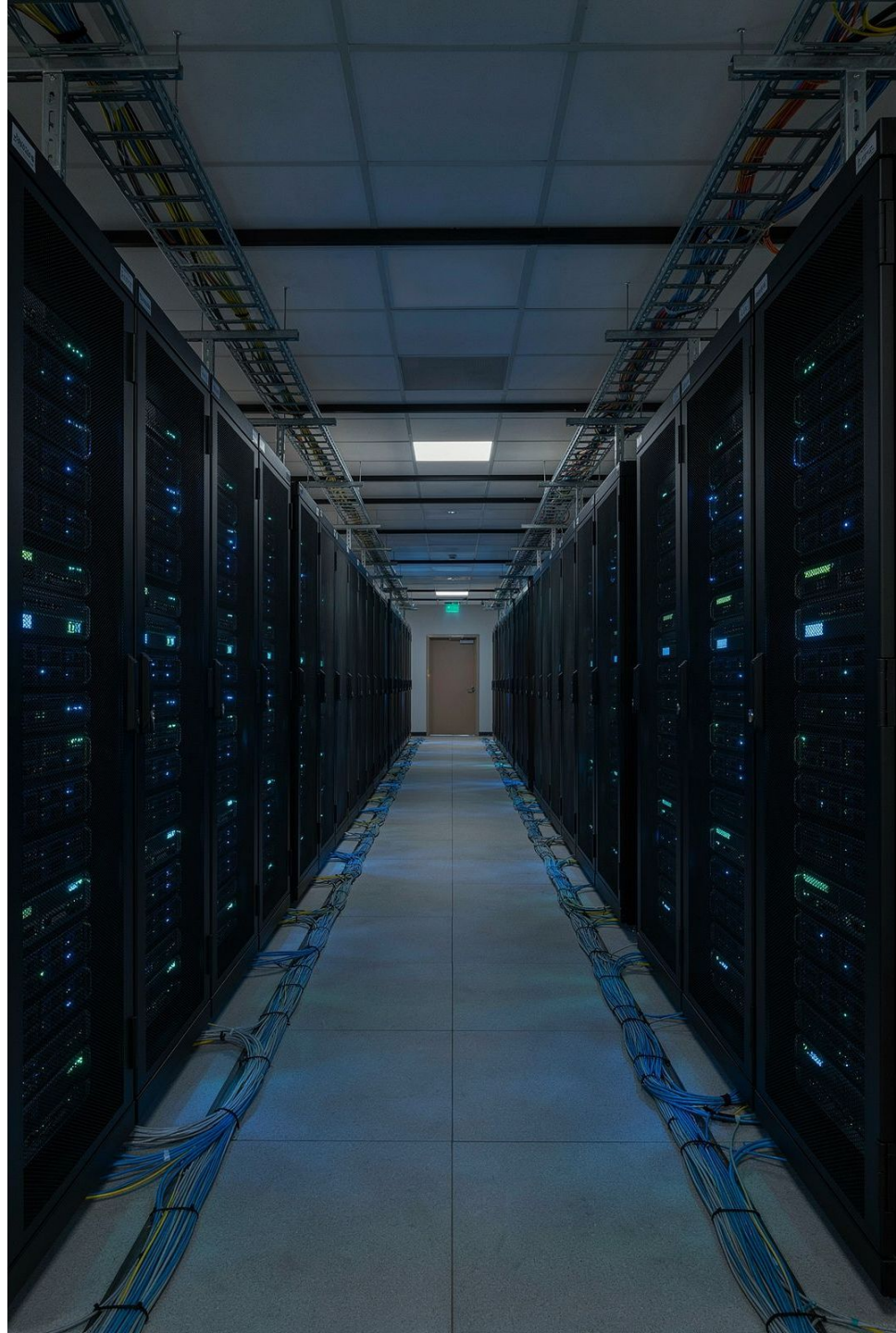
SPRINT 3 · DEL DATO AL SERVICIO · S03.1

Bases de datos relacionales y SQL

Los datos de TurisData viven en turisdata.db: tres tablas relacionadas. SQL es el idioma con el que se les hacen preguntas, y se usa en casi todos los proyectos de IA.



El modelo relacional de TurisData

Tabla	Una fila es…	Clave
alojamientos	un hotel, apartamento o casa rural	id_alojamiento
clientes	una persona (con email anonimizado)	id_cliente
reservas	una reserva de un cliente en un alojamiento	id_reserva, más claves foráneas a las otras dos

- Clave primaria: identifica una fila de forma única.
- Clave foránea: apunta a la clave primaria de otra tabla; así no se repiten datos y se evitan errores («UK» / «uk» / «Reino Unido»).
- Los datos son sintéticos: no son estadísticas reales del turismo canario.

SELECT, WHERE, ORDER BY, LIMIT

```sql
SELECT nombre, categoria, plazas
FROM alojamientos
WHERE isla = 'Tenerife' AND plazas > 100
ORDER BY plazas DESC
LIMIT 5;
```

1. FROM: qué tabla.
2. WHERE: qué filas.
3. SELECT: qué columnas.
4. ORDER BY y LIMIT: orden y recorte.

Cada consulta termina en punto y coma. En DB Browser: selecciona una consulta y ejecútala con Ctrl+Intro.

JOIN: unir tablas

```
SELECT r.id_reserva, c.pais AS pais_cliente, a.isla, r.canal
FROM reservas r
JOIN clientes c     ON c.id_cliente = r.id_cliente
JOIN alojamientos a ON a.id_alojamiento = r.id_alojamiento
LIMIT 5;
```

- JOIN (interno): solo las filas con pareja en ambas tablas.
- LEFT JOIN: conserva todas las de la izquierda; lo que no tiene pareja sale NULL.
- Clientes sin reservas: LEFT JOIN … WHERE r.id_reserva IS NULL.

GROUP BY y HAVING

```
SELECT canal, COUNT(*) AS reservas,
       ROUND(100 * AVG(cancelada), 1) AS pct_cancelacion
FROM reservas
GROUP BY canal
HAVING COUNT(*) > 500
ORDER BY pct_cancelacion DESC;
```

- COUNT, SUM, AVG, MIN, MAX resumen grupos.
- WHERE filtra filas antes de agrupar; HAVING filtra grupos después.
- AVG(cancelada) con 0/1 es directamente la tasa de cancelación: la pregunta del cliente.

Subconsultas y CTE

```
WITH por_canal AS (
    SELECT canal, AVG(cancelada) AS tasa FROM reservas GROUP BY canal
)
SELECT canal, ROUND(100 * tasa, 1) AS pct
FROM por_canal
WHERE tasa > (SELECT AVG(cancelada) FROM reservas);
```

- Escalar: devuelve un valor (WHERE noches > (SELECT AVG(noches) ...)).
- Con IN: devuelve una lista de ids.
- CTE (WITH): pone nombre a un paso intermedio y la consulta se lee de arriba abajo.

CRUD y transacciones

```
INSERT INTO reservas_practica (id_reserva, id_cliente, id_alojamiento, noches, canal)
VALUES (900001, 1, 1, 3, 'directo');          -- Create (nombra las columnas)

SELECT id_reserva, precio_noche FROM reservas_practica WHERE id_reserva = 900001;
UPDATE reservas_practica SET precio_noche = 135.0 WHERE id_reserva = 900001;   -- Update

BEGIN;  DELETE FROM reservas_practica WHERE canal = 'agencia';  ROLLBACK;      -- Delete, deshecho
```

Sin WHERE, UPDATE y DELETE afectan a TODA la tabla. Primero SELECT, después UPDATE con el mismo WHERE. Y se practica siempre en una copia (reservas_practica), nunca en la original.

Modelado, restricciones e índices

```
CREATE TABLE p_reservas (
    id_reserva   INTEGER PRIMARY KEY,
    pais         TEXT REFERENCES p_paises(codigo),   -- clave foránea
    noches       INTEGER NOT NULL CHECK (noches >= 1),
    precio_noche REAL    NOT NULL CHECK (precio_noche > 0)
);
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM reservas WHERE id_cliente = 100;
```

- Las restricciones (PRIMARY KEY, REFERENCES, CHECK) hacen que el dato malo falle al entrar.
- Un índice acelera búsquedas: SCAN recorre todo; SEARCH … USING INDEX va directo.
- Administración mínima: copias de seguridad, permisos y no tocar producción.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

ACTIVIDAD · Scripts SQL 01–06 en DB Browser (sprints/S03/actividades/sql/), en este orden: SELECT, JOIN, GROUP BY y subconsultas; después, CRUD sobre una copia y modelado e índices — 100 min.